

LEBRE v0.51 Architecture Specification

Minimal, cheap, competitive and explainable online forecasting

LEBRE: Lifecycle-governed Evidence-Based Resource Evolution

Architecture: structural expert + memory expert + dynamic model averaging

COST

~121 operations/step; 0.4–3 KB depending on inputs and period

ACCURACY (HELD-OUT)

0.84× NLinear · 1.37× Chronos-2

START-POINT SENSITIVITY

max. variation 1.08× across 10 starts

STATUS

research version promoted by pre-registered evaluation

1. Executive summary

LEBRE is an **online** time-series forecaster: at every step it receives an observation, predicts the next value, learns from the error and moves on, without re-processing the past. Version **v0.51** has four parts: a **structural expert** (input-driven dynamics, with structure discovered by statistical tests), a **memory expert** (level and cycle of the target itself), a **combiner** based on dynamic model averaging, and an adaptive **prediction interval** with a 90% coverage target.

Premise: LEBRE does **not** aim to be the most accurate model. Its goal is **good, competitive accuracy at minimal computational cost** (sized for embedded devices), **without per-series tuning**, and with **explainability and observability** of every forecast. Foundation models and classical models fitted series by series can be more accurate; this document shows where and by how much.

Main result (pre-registered evaluation on 10 real series never used during development, 5 of them Brazilian): best of the pre-registered online comparators, with error 0.84× that of the calibrated online NLinear; 1.37× that of Chronos-2 (foundation model), at $\sim 10^6$ – 10^7 times lower cost; 121 operations per step on average (peak 196); ~ 1.3 KB of state (median; 0.4–2.1 KB depending on the number of inputs and the period, §7); maximum variation of 1.076× depending on the start point.

What the post-hoc analyses showed (§10–§12): on real series, **almost all of the result comes from the memory expert**. The memory alone is at 1.034× the full LEBRE and gives identical results on 8 of 10 series. A classical airline model **fitted series by series by maximum likelihood** is ~ 5 – 10% more accurate than LEBRE. What LEBRE adds to the memory is universality (input-driven systems), named structure discovery, no per-series tuning and adaptive intervals; the structural part still has no evidence on real, genuinely input-driven series. On synthetic processes, structure discovery is reliable for **one delay**, partial for **several delays** and **fails** when a latent state is driven by an input, and the first promotion usually takes 1.5–3 thousand steps (§11.2).

Document guide

1. Executive summary	§1
2. What LEBRE is, what it is for and its premise	§2
3. Design principles	§3
4. Architecture overview	§4
5. Mathematical formulation	§5
6. One step and causality	§6
7. Computational cost	§7
8. Observability and explainability	§8
9. Empirical validation and methodological rigour	§9
10. LEBRE versus the memory alone and strong classical baselines	§10
11. Statistical guarantees and reach of structure discovery	§11
12. What this document demonstrates and what it does not	§12
13. Evolution of the architecture and lessons	§13
14. Limits and claims we do not make	§14
15. Parameters	§15
16. References	§16

2. What LEBRE is, what it is for and its premise

The problem

Many real systems produce measurements as a continuous stream — energy load, wind and solar generation, flow, air quality, industrial sensors, exchange rates — and need a forecast of the next value *on the device itself*, with scarce memory and energy, no GPU and no possibility of offline training on large histories. There are three obstacles: (i) the most accurate modern models (transformers and foundation models) require hundreds of megabytes and billions of operations per forecast; (ii) classical statistical models are cheap but must be fitted series by series; (iii) in critical applications, a forecast without explanation and without quantified uncertainty has little operational value.

The premise

LEBRE **does not compete to be the most accurate model**. It is designed to occupy a specific point of the accuracy–cost–transparency trade-off: **competitive accuracy** (at the level of modern lightweight models), **minimal cost** (hundreds of operations per step and a few KB), **no per-series tuning** (the same constants on any stream), and **explainability and observability** of every forecast. Losing a few percentage points of accuracy to a foundation model or to a classical model fitted series by series is acceptable within this premise; losing competitiveness, low cost or transparency is not.

What LEBRE does

- **Learns online, at every step**, with no separate training phase and no per-series calibration.
- **Works on two kinds of problem**: series driven by their own memory (seasonality, level) and systems driven by inputs (delays, hidden states), deciding by itself which hypothesis holds at each moment.
- **Decomposes every forecast** into named contributions and **logs** when and why each structure entered or left the model.
- **Reports its own uncertainty** with an adaptive interval targeting 90% coverage (§5.5).
- **Costs ~100–150 floating-point operations per step** and 0.4 to ~3 KB of state, depending on the number of inputs and the seasonal period (§7). This budget is compatible with microcontrollers, but execution on embedded hardware has **not been verified**, and FP does not measure energy.

What it is for (typical use cases)

Embedded short-term forecasting in energy meters and controllers; monitoring of renewable generation; industrial sensors; IoT streams; any context where the forecast must be cheap, auditable, accompanied by uncertainty, and where fitting a model for each series is not viable.

What LEBRE is not

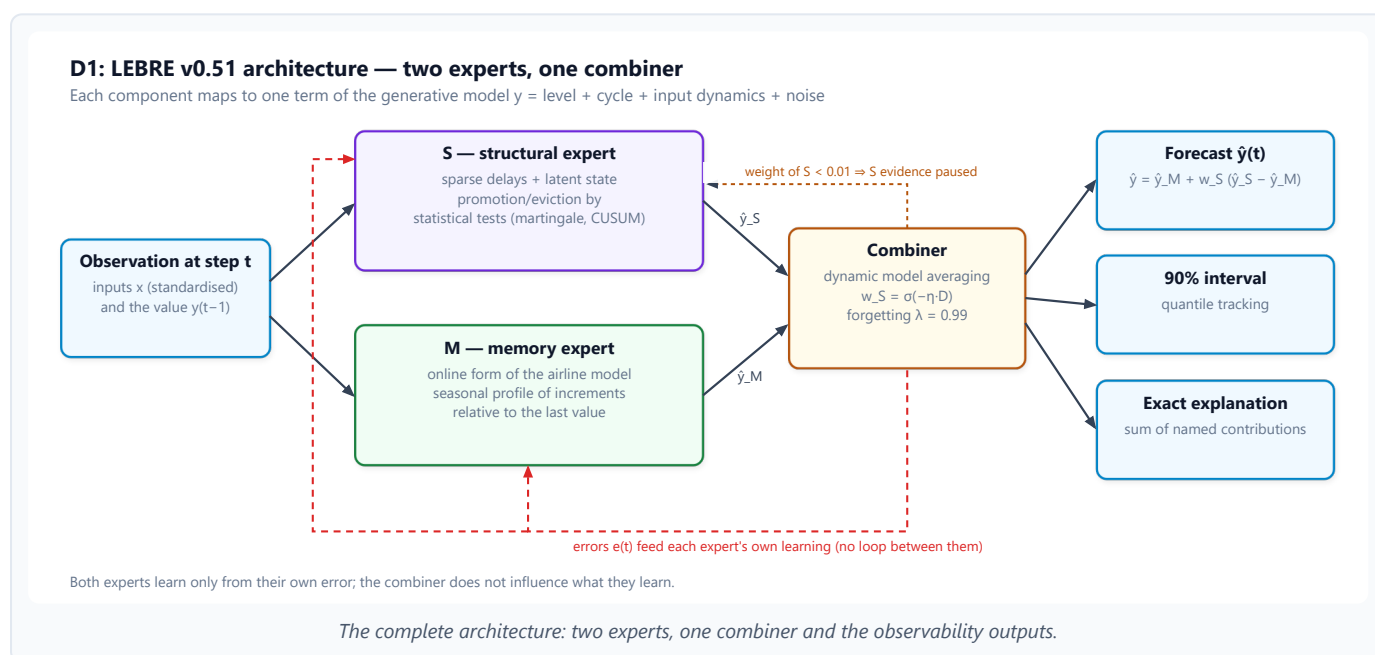
It is not a foundation model nor a long-horizon forecaster; it does not model strongly non-linear input–target relations (the class is linear in the parameters); it does not claim physical causality (explanations describe the model, i.e. predictive relevance). The name comes from the project's original family of architectures (*Lifecycle-governed Evidence-Based Resource Evolution*): structures enter and leave the model by statistical evidence, under an explicit resource budget.

3. Design principles

LEBRE v0.51 was designed from four requirements and four operational principles inherited from earlier versions:

Principle	Meaning in v0.51
Cost	≤ 150 floating-point operations per step on average and peak ≤ 500 (up to ~ 6 inputs); a few KB of memory.
Minimal architecture	Every component maps to a term of the generative model and was kept only if removing it made results worse (§9.6).
Competitiveness	Accuracy at the level of modern lightweight models (NLinear, DLinear) and close to foundation models.
Observability and explainability	Exact decomposition of each forecast, interpretable weights, adaptive interval, change alarms and accounted cost.
P2	Stream-time evidence: evidence accrues at a fixed rate per step, not per event.
P4	Silence is not evidence: without input excitation no decision statistic is updated.
P6	Input contract: inputs standardised causally by the environment (§9.1); a value outside the contract ($ x > 8$) is used clipped in the forecast but produces neither evidence nor learning.
No calibration	No hyper-parameter is tuned per series; the only information from the environment is the sampling period s (e.g. 24 for hourly data).

4. Architecture overview



Why two experts. Real seasonal series and input-driven systems call for opposite representations. In the former, the decisive information is in the target's own past, and the forecast should be made relative to the last value. In the latter, anchoring on the last value injects noise, and what matters are the inputs and their delays. During development, attempts to merge both representations into a single model (cascade, joint learning or regression combination) failed in at least one of the two worlds (§9.6). The minimal architecture therefore has two experts and a combiner that tracks which hypothesis currently holds.

5. Mathematical formulation

5.1 Generative model

We start from the structural decomposition of time series (Harvey, 1989):

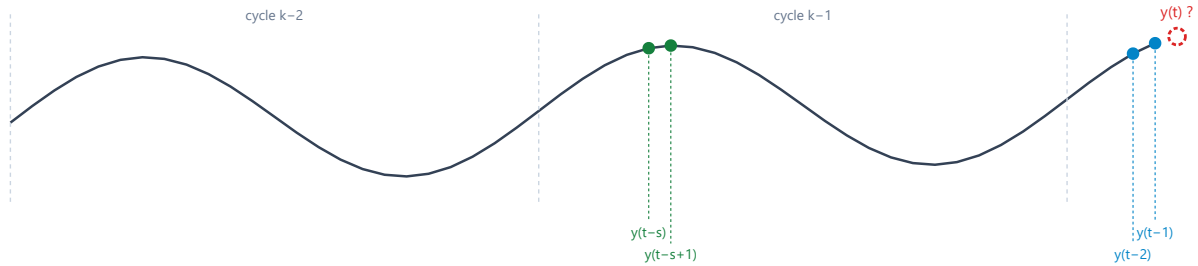
$$y_t = \ell_t + c_{t \bmod s} + f(\mathbf{x}_{t-1}, \mathbf{x}_{t-2}, \dots) + \eta_t$$

where ℓ_t is a slowly varying level, c is a cycle of period s , f is the input-induced dynamics (delays and hidden states) and η_t is noise, possibly with short autocorrelation. State-space exponential smoothing (Hyndman et al., 2008), the Box–Jenkins airline model (1970) and Holt–Winters (Holt, 1957; Winters, 1960) are special cases. Recent lightweight methods use the same idea of separating the cycle and forecasting the rest: CycleNet (Lin et al., 2024), SparseTSF (Lin et al., 2024), FITS (Xu et al., 2024) and the winner of the M4 competition (Smyl, 2020).

5.2 Memory expert M (level and cycle)

D3: Memory expert — what it reads from the past

All features are differences relative to the last value: there is no level to learn



$\hat{y}_t^M = y_{t-1} + w_1[m - y_{t-1}] + w_2[y_{t-1} - y_{t-2}] + w_3[y_{t-1} - y_{t-2}] + w_4[y_{t-s+1} - y_{t-s}] + w_5 G[\text{phase}]$
 m : exponential mean of the target (mean reversion $\Leftrightarrow$ simple exponential smoothing with a learned α)
 $G[\text{phase}]$: exponential average (factor 0.9) of the increments observed at this same phase in previous cycles
 $\Leftrightarrow$ seasonal increment forecast by the airline model SARIMA(0,1,1)(0,1,1) $_s \Leftrightarrow$ Holt–Winters seasonal state
 $w_1, \dots, w_5$: learned online by NLMS (scale invariant); without a declared period only $[m - y_{t-1}, y_{t-1} - y_{t-2}]$ are used

The five memory features, all relative to the last value.

$$\begin{aligned} \hat{y}_t^M &= y_{t-1} + \mathbf{w}^\top \phi_t, & \phi_t &= [m_{t-1} - y_{t-1}, y_{t-1} - y_{t-2}, y_{t-s} - y_{t-1}, y_{t-s+1} - y_{t-s}, G_{t \bmod s}] \\ G_p &\leftarrow G_p + a [(y_t - y_{t-1}) - G_p], & a &= 0.1; & m_t &\leftarrow m_{t-1} + 0.01 (y_t - m_{t-1}) \\ \mathbf{w} &\leftarrow \mathbf{w} + \frac{\mu (y_t - \hat{y}_t^M)}{\|\phi_t\|^2} \phi_t, & \mu &= 0.05 \end{aligned}$$

Why these features. (i) The airline model SARIMA(0,1,1)(0,1,1) $_s$ forecasts the seasonal increment as an exponentially weighted average of the increments at the same phase in previous cycles, with factor Θ ; the profile G with $a = 0.1$ is exactly that with $\Theta = 0.9$, and coincides with the Holt–Winters seasonal state and with CycleNet's "learnable recurrent cycle". (ii) With weight w_1 , the mean-reversion feature yields $y_{t-1} + w_1(m - y_{t-1})$, i.e. simple exponential smoothing with a learned constant — optimal for a noisy local level (Muth, 1960). (iii) The recent increment captures short autocorrelation, and the two seasonal-lag features carry the raw information of the last cycle. (iv) Forecasting relative to the last value removes the need to learn the level, which caused the instability of earlier versions (§13); this is also NLinear's normalisation (Zeng et al., 2023).

Why NLMS. NLMS (Nagumo and Noda, 1967) is the minimum-norm step that zeroes the a-posteriori error along the feature direction; LMS is H_∞ -optimal (Hassibi, Sayed and Kailath, 1996), i.e. it minimises the worst-case ratio of error energy to

disturbance energy without distributional assumptions — the right property for non-stationary streams. NLMS is also *scale invariant* in its features, so the target needs no standardisation. RLS would converge faster but costs $O(k^2)$.

Numerical contract. The update of M has *no* regularisation term in the denominator: it is skipped only when $\|\phi_t\|^2 = 0$ exactly. This keeps exact scale invariance but leaves one case unprotected: after a long, exactly constant stretch the recent increments are zero or near zero, and a large jump produces a huge step. In a synthetic "constant then step" test, the weights of M diverged. This case did not occur on any of the real series evaluated, but series with long plateaus and jumps (e.g. quantised sensor signals) are outside this version's numerical contract. Expert S uses a fixed term $\varepsilon = 10^{-6}$ in the denominator; since its inputs are standardised, the effect on invariance is negligible.

5.3 Structural expert S (input-driven dynamics)

$$\hat{y}_t^S = \theta_0 + \theta_b^\top \mathbf{x}_t + \sum_{a \in \mathcal{A}_t} \theta_a \varphi_a(t), \quad |\mathcal{A}_t| \leq 4$$

The dense base reads the current inputs; the active set $\mathcal{A}_t$ contains physically meaningful structural atoms: *delays* $x_{i,t-k}$ with $k \leq 32$, a *latent state* and a *drive* of the latent state. The latent state implements the steady-state Kalman predictor in innovations form: for a first-order hidden state, the optimal predictor is linear in first-order filters of the innovation and of the input with pole p ; LEBRE uses a pole bank $\{0, 0.5, 0.8, 0.95\}$:

$$s_t = p s_{t-1} + (1-p) u_t / \hat{\sigma}, \quad q_{i,t} = p q_{i,t-1} + (1-p) x_{i,t}$$

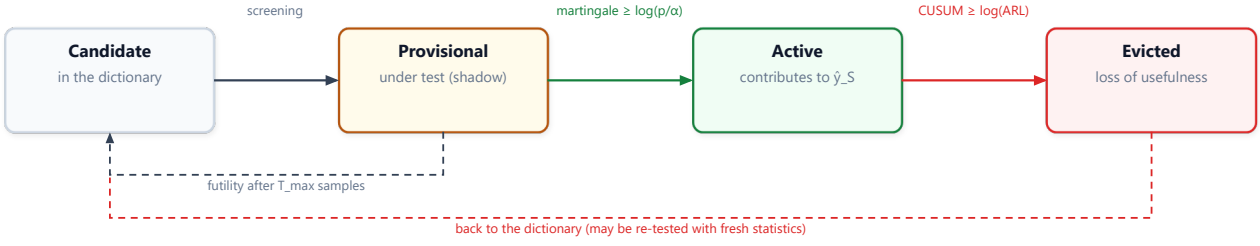
All parameters are learned by NLMS with S 's own error. **Promotion:** every candidate is tested in shadow (without entering the forecast) by a self-normalised mixture martingale; it is promoted when

$$\log M_t = \frac{\tau S_t^2}{2(1 + \tau Q_t)} - \frac{1}{2} \log(1 + \tau Q_t) \geq \log(p_{\text{dict}}/\alpha), \quad S_t = \sum e_k \varphi_k, \quad Q_t = \sum (e_k \varphi_k)^2$$

With conditionally symmetric increments, M_t is a supermartingale (de la Peña, 1999; Howard et al., 2021), and by Ville's inequality (1939) the probability of a false promotion at any time is $\leq \alpha/p$ per candidate *and per test episode*. The control holds per test, not for the whole stream: since candidates can be re-tested with fresh statistics, false promotions can accumulate over very long streams (the empirical rate is in §11). **Eviction:** a CUSUM (Page, 1954) of the log-likelihood ratio "without vs. with" the atom, plus a parsimony rent $r = h/T_{\text{idle}}$ in the MDL spirit (Rissanen, 1978), removes the atom when it crosses $h = \log(\text{ARL}) = \log 6000$; *without* rent, the mean time to a false alarm would be $\geq e^h$ (Lorden, 1971; Moustakides, 1986). **The rent breaks that guarantee:** it pushes the CUSUM towards eviction, so an eviction means "the atom stopped paying for itself", whether because of a real change or because of a small contribution. Evictions are therefore treated as **loss-of-usefulness signals**, not as alarms with a controlled error rate (empirical rate in §11).

D4: Structural lifecycle of expert S

Promotion: test with type-I error controlled per episode. Eviction: CUSUM with rent (no formal ARL guarantee)



No evidence is accrued when:

- **silence:** the power of the inputs read is < 10% of the long-run power (P4: silence is not evidence);
- **quarantine:** a clipped value ($|x| > 8$) is readable by some lag (P6: an input outside the contract is not evidence);
- **dormancy:** the weight of S in the combiner is < 0.01 (S's structure does not affect the forecast). Prediction and weights continue.

Structural lifecycle and the three conditions under which no evidence is accrued.

Operational details of search and tests

- **Dictionary.** Delays $x_{i,t-k}$ with $1 \leq k \leq L = 32$ for each input, 4 latent states (one per pole) and d drives: $p_{\text{dict}} = dL + 4 + d$ (169 with $d = 5$, threshold ≈ 8.1 nats).
- **Screening.** At every step, 2 dictionary delays are probed in rotation; each one's score is an exponential average (weight 0.2) of $e_t x_{i,t-k}$. Every 10 steps, the free test slots (at most 4 delays under test at once) receive the delays with the highest score.
- **Test.** Each candidate under test receives one evidence sample every 2 steps; decisions are made every 10 steps; after $T_{\text{max}} = 200$ samples without crossing the threshold, the test is stopped for futility and the candidate returns to the dictionary.
- **Mixture prior.** $\tau = \rho / (\hat{\sigma}^2 m_\varphi)$, with $\rho = 1$ (unit-information prior) and m_φ the regressor's design power (1 for standardised inputs and their lags).
- **Latent state.** At most **one** latent state is active. While none is active, one candidate per pole is tested in parallel; the first to cross the threshold is promoted and the other poles are no longer tested. Only then are the d drives tested, always with the pole of the promoted latent.
- **Budget.** At most $M_{\text{max}} = 4$ active structural atoms (latent and drive count; the dense base does not). If a candidate crosses the threshold with a full budget, it replaces the active atom with the largest CUSUM, provided that CUSUM is $\geq h/2$; otherwise the promotion is deferred.

Three evidence gates, one justification. Silence, quarantine and dormancy are rules that depend only on the past (they are *predictable*). Skipping observations by a predictable rule preserves the supermartingale property of the evidence statistics (Ramdas et al., 2023), so each test's error control remains valid. **Quarantine** is a hard rejection of leverage points, in the spirit of Mallows GM-estimators (Hampel et al., 1986). **Dormancy** follows the *sleeping experts* idea (Freund et al., 1997): while S has negligible weight its structure does not affect the forecast, and only structural discovery is paused. Prediction, weight learning and the scale $\hat{\sigma}^2$ continue, so S can regain weight. **Cold start:** when a latent state is promoted, the drive states start at zero. The test remains valid for any predictable regressor, and the peak cost drops from ~ 500 to $O(d)$.

5.4 Combiner: dynamic model averaging

$$D_t = \lambda D_{t-1} + \frac{(y_t - \hat{y}_t^S)^2 - (y_t - \hat{y}_t^M)^2}{\hat{\sigma}_t^2}, \quad w_t^S = \frac{1}{1 + e^{\eta D_t}}, \quad \hat{y}_t = \hat{y}_t^M + w_{t-1}^S (\hat{y}_t^S - \hat{y}_t^M)$$

With two experts, the weights of dynamic model averaging with forgetting (Raftery, Kárný and Ettler, 2010) depend only on the accumulated loss difference. With a Gaussian predictive likelihood and losses normalised by the variance $\hat{\sigma}^2$ of the combined error, the exponent is $\eta = 1/2$ — a derived value, not a tuned one. The factor $\lambda = 0.99$ (~ 100 -step memory) lets the combiner track regime changes, as in *fixed-share* (Herbster and Warmuth, 1998); the algorithm belongs to the family of exponentially weighted aggregators with regret guarantees (Vovk, 1990; Cesa-Bianchi and Lugosi, 2006). Regression combination (Granger and

Ramanathan, 1984) was tested and discarded: it requires holding a weight near 1 on a high-variance feature, and NLMS misadjustment turns into noise (§9.6).

5.5 Prediction interval

$$\hat{q}_t = \max(0, \hat{q}_{t-1} + \gamma \hat{\sigma}_t (1\{|e_t| > \hat{q}_{t-1}\} - \alpha)), \quad \alpha = 0.1, \gamma = 0.1, \quad \text{interval} = \hat{y}_t \pm \hat{q}_t$$

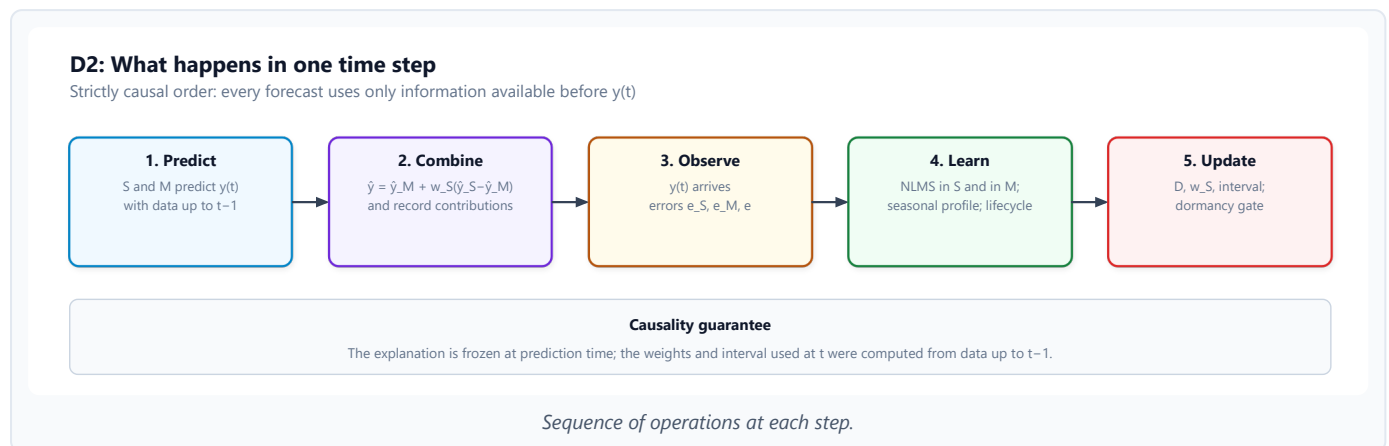
With one update per step, quantile tracking (Gibbs and Candès, 2021; Angelopoulos, Candès and Tibshirani, 2023) guarantees long-run coverage close to 90% without distributional assumptions. **v0.51 does not use exactly that form:** to reduce cost, the square root, the inverse and the quantile are updated only every 8 steps, and the update uses only the error indicator *of the step at which it happens*, with an 8× step. The mean rate is preserved, but the guarantee above does not fully carry over: when the seasonal period is a multiple of 8 (24, 48, 144), the updates only observe some phases of the cycle (3 of 24, for example), and coverage then depends on which phases are sampled. On the real series evaluated, per-series coverage ranged from 0.85 to 0.97. In a later analysis, accumulating the 8 indicators of each block (same cadence, practically the same cost) brought every series to 0.900–0.906; that variant is *not* part of v0.51.

5.6 Exact additive explanation

$$\hat{y}_t = w^S \left[\theta_0 + \sum_i \theta_{b,i} x_{i,t} + \sum_{a \in \mathcal{A}_t} \theta_a \varphi_a(t) \right] + (1 - w^S) \left[y_{t-1} + \sum_{k=1}^5 w_k \phi_{k,t} \right]$$

Every term has a name and a value, and the sum equals the forecast up to machine rounding. It is an *intrinsic* explanation (the model is the explanation; Rudin, 2019), not a post-hoc approximation.

6. One step and causality



The two experts never see the combined forecast or the combined error; each learns only from its own error. The combiner therefore creates no feedback loop between them — a cascade and joint learning, tested during development, were unstable precisely because of such a loop.

7. Computational cost

We count every floating-point addition, multiplication, division and comparison executed by the algorithm (FP). Operations done every k steps are divided by k . FP is not energy, but it is a reproducible, language-independent indicator.

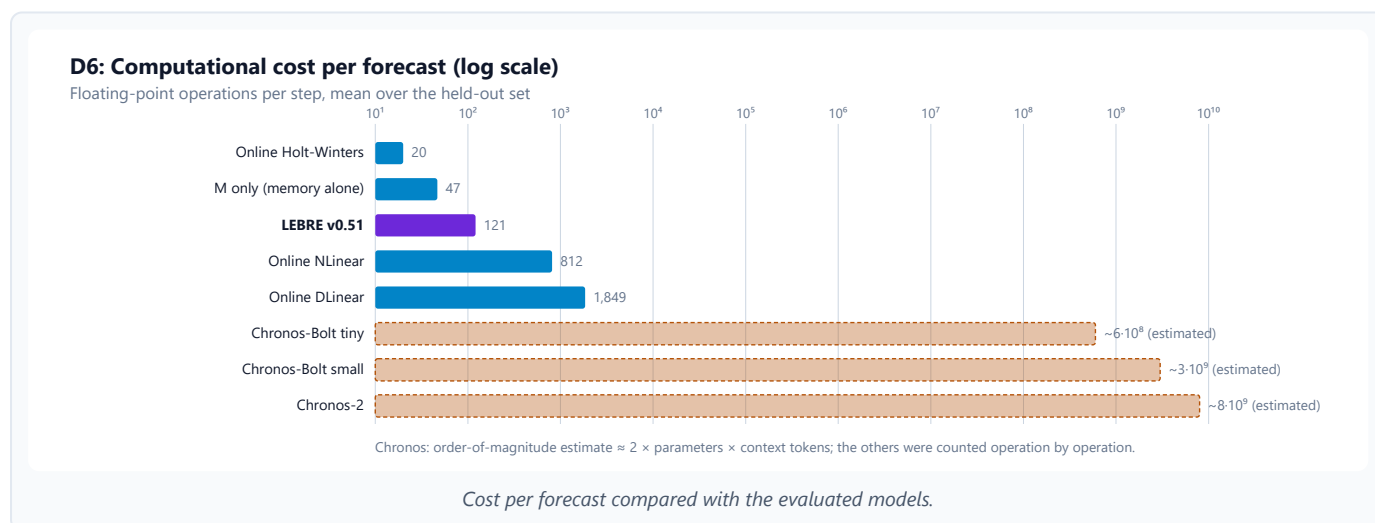
Component	FP/step	Composition
S — structural expert ($d = 5$)	~85–100	dense base ~6d; atoms, latent, search and evidence
S in dormancy (seasonal series)	~60–75	no tests, CUSUM or evidence updates
M — memory (5 features)	~40–47	differences, inner product, NLMS, profile, mean
M without period (2 features)	~22	—
Combiner	~17	errors, loss difference, 1 exp every 8 steps
Interval	~2	comparison and update every 8 steps

Verified budget. On the 14 internal tasks ($d = 5$) the maximum per-task mean was 147 FP and the peak 249; on the 10 held-out series ($d \leq 6$), 138 and 196. Cost grows by ~8 FP per extra input (S's dense base); with 25 inputs we measured ~260 on average and ~520 at peak — that is the budget boundary.

What the count does not include. Causal standardisation of the inputs (§9.1) is done by the evaluation environment, identically for every model, and is not included in the count above. On a device it would cost about 12 FP per input per step (~60–70 FP with 5–6 inputs).

Memory. The state grows with the number of inputs d (lag history of S) and with the period s (buffer and seasonal profile of M). In single precision the state is approximately $196d + 12s + 190$ bytes (no period: $196d + 180$); in Python, which uses double precision, twice that. The “~1.3 KB” figure is the median over the held-out series, not a bound:

Inputs d	No period	$s = 24$	$s = 144$	$s = 1440$	Largest s with ≤ 1.3 KB (1,331 B)
1	0.4 KB	0.7 KB	2.1 KB	17 KB	78
3	0.8 KB	1.0 KB	2.4 KB	18 KB	46
5	1.1 KB	1.4 KB	2.8 KB	18 KB	13
6	1.3 KB	1.6 KB	3.0 KB	18 KB	none



Model	FP/step (mean)	FP peak	Model memory	ms/step (CPU)
LEBRE v0.51	121	196	1.32 KB	0.072
Structural expert alone	98	781	0.44 KB	0.054
Dense-window variant	1,022	2,459	2.29 KB	0.155
Online NLinear	812	2,022	0.75 KB	0.026
Online DLinear	1,849	4,614	1.12 KB	0.051
Online Holt-Winters	20	20	0.11 KB	0.014
Chronos-Bolt tiny	$\sim 6 \cdot 10^8$ (est.)	—	~ 36 MB	0.778
Chronos-Bolt small	$\sim 3 \cdot 10^9$ (est.)	—	~ 190 MB	3.171
Chronos-2	$\sim 8 \cdot 10^9$ (est.)	—	~ 480 MB	15.593
Chronos-2 w/ covariates	$\sim 10^{10}$ (est.)	—	~ 480 MB	50.887

Wall-clock times in Python on a single CPU core (LEBRE and linear models) and in PyTorch with 16 threads (Chronos); in a compiled language LEBRE would be in the microsecond range. Memory of the Chronos models: ~ 36 MB (Bolt tiny, 9 M parameters), ~ 190 MB (Bolt small) and ~ 480 MB (Chronos-2, 120 M parameters) in float32.

8. Observability and explainability

At every step LEBRE v0.51 delivers, besides the forecast:

- **Exact decomposition** of the forecast into named contributions (§5.6).
- **Weight w_S** : tells whether the system behaves as "input-driven" ($w_S \approx 1$) or "driven by its own memory" ($w_S \approx 0$).
- An **adaptive interval** with a 90% coverage target (§5.5).
- **Lifecycle log**: when and why each structure entered or left, with the evidence in nats. Promotions have type-I error controlled per test episode; evictions are loss-of-usefulness signals, *without* a guaranteed false-alarm rate (§5.3, §11).
- **Accounted cost** per component.

What is and what is not a differentiator. Exact decomposition is a trivial property of any linear additive model: it is true and useful, but it does not distinguish LEBRE. What LEBRE adds is (i) the *selection* of the contributions by statistical evidence, with physical names (which delays and which latent state matter), (ii) the auditable log of those decisions and (iii) the weight w_S , which tells which hypothesis holds. On seasonal series, where the memory dominates, the real explanation is simple: *last value, corrected by the typical seasonal increment of this phase* — as example 1 shows.

Real example 1 — seasonal series (national grid load, last held-out step)

Forecast 59,707.0 MWavg (observed 60,217.7), interval $\pm 1,747.5$, structural weight $w_S = 0.000$: memory dominates. Contributions (largest in magnitude; the sum of all of them is exactly the forecast):

Contribution	Value
Memory (weight 1.00): last value $y(t-1)$	63,349.5
Memory (weight 1.00): seasonal profile of increments	-2,496.2
Memory (weight 1.00): recent increment	-754.9
Memory (weight 1.00): increment one cycle ago	-217.8
Memory (weight 1.00): seasonal: $y(t-24)$	-186.7
Memory (weight 1.00): mean reversion	13.176

Example 2 — synthetic input-driven task (delay + latent state)

Forecast -0.520 ± 0.916 (observed 0.025), $w_S = 1.000$: the structural expert dominates and the explanation names the recovered structure. First lifecycle events: $t=1400$: promoted — x_3 delayed 12 steps; $t=1650$: promoted — latent state ($\tau \sim 2$ steps, pole 0.50); $t=1990$: promoted — x_0 drives the latent state (pole 0.50); $t=2330$: evicted — latent state ($\tau \sim 2$ steps, pole 0.50); $t=2330$: evicted (coupled) — x_0 drives the latent state (pole 0.50); $t=2650$: promoted — latent state ($\tau \sim 5$ steps, pole 0.80).

Contribution	Value
Structural (weight 1.00): x_3 delayed 12 steps	-0.855
Structural (weight 1.00): x_0 drives the latent state (pole 0.80)	0.251
Structural (weight 1.00): current inputs (base)	0.108
Structural (weight 1.00): bias	-0.061
Structural (weight 1.00): latent state ($\tau \sim 5$ steps, pole 0.80)	0.038

9. Empirical validation and methodological rigour

9.1 Protocol and safeguards against bias

- **Pre-registration.** Hypotheses, criteria, thresholds, datasets, comparators and seeds were written before any run, and the document's SHA-256 hash was recorded. The model code was frozen by hash, and the run checks both hashes before starting.
- **Development/evaluation separation.** All design choices were made on previously used development data; the main evaluation set (held-out) consists of 10 series that no LEBRE version had seen. Random seeds verified never to have been used.
- **Causal prequential evaluation.** At every step the model predicts and only then sees the value; inputs are standardised causally with exponential means and variances (factor 10^{-4} , ~10,000-step memory, initialised at mean 0 and variance 1), updated only after each forecast; error is measured from 30% of the series onward. **NMSE** = mean squared error / test-segment variance. Since all comparisons use ratios *within the same series*, the denominator cancels: the geometric mean of NMSE ratios equals that of MSE ratios. We also report *skill* and MAE relative to persistence (§9.5).
- **Advantage given to comparators.** NLinear, DLinear, Holt-Winters and IPNLMS had hyper-parameters chosen per series on the first min(15%, 5000) steps. LEBRE was *not* calibrated. Online Holt-Winters with constants from a grid turned out to be a weak comparator (worse than persistence on one series); §10 therefore adds an airline model fitted by maximum likelihood.
- **Foundation models.** Chronos-Bolt tiny was evaluated on the whole test segment; because of CPU cost, Chronos-Bolt small and Chronos-2 on 1000 evenly spaced points and Chronos-2 with covariates on 500 (mean error estimated on those evenly spaced points of the test segment). Same information set (values up to $t-1$; context 512, or 256 with covariates). Public series may be in these models' pre-training data, which would favour them.
- **Start-point sensitivity.** Each series was re-run from 10 different start points (0, 50, ..., 450 steps dropped) with the same test window. This measures how much the result depends on initialisation and warm-up; it does *not* measure stability across different stretches of the series (seasons, years, regimes), which was not evaluated.
- **Earlier LEBRE versions.** They are not in the ranking. They appear only as *ablations with descriptive names*, when they answer a design question: the *structural expert alone* (earlier version without memory) shows why the memory is needed; the *dense-window variant* (earlier version with 96–288 lags, ~1000 FP/step) shows that the simplification did not lose accuracy — it was a pre-registered criterion.
- **Decision rule fixed in advance.** Promotion required passing *all* binding criteria. The analyses in §10–§11 were done *after* the evaluation and are labelled post-hoc; they do not change the pre-registered decision. The pre-registration and hashes are internal project records and do not accompany this document.

9.2 Held-out set (10 real series never used)

	Series	Frequency	d	s
Q1	ONS — load of the Brazilian National Grid (2019–20)	hourly	6	24
Q2	ONS — hydro generation, Southeast/Center-West (2019–20)	hourly	6	24
Q3	ONS — hydro generation, South (2019–20)	hourly	6	24
Q4	ONS — wind generation, National Grid (2019–20)	hourly	6	24
Q5	Central Bank of Brazil — GBP/BRL (2005–24)	daily	1	—
Q6	Australian electricity demand, series 4	30 min	1	48
Q7	Pedestrian counts (Melbourne), series 4	hourly	1	24
Q8	Solar power (USA), series 4	10 min	1	144
Q9	Air quality (KDD Cup 2018), series 4	hourly	1	24
Q10	Australian electricity demand, series 5	30 min	1	48

d = number of input variables (all variables at $t-1$, including the target itself); s = declared period. Six of the ten series have d = 1, i.e. **no exogenous input**: there the structural hypothesis only has the target's own past. Sources: Brazilian National Electric System Operator (open data), Central Bank of Brazil (SGS) and the Monash Time Series Forecasting Archive (Godahewa et al., 2021).

9.3 Pre-registered criteria and outcome

Criterion	Threshold	Result
Mean cost per task (internal d = 5; held-out d ≤ 6)	≤ 150 FP	147 / 138 ✓
Peak per step	≤ 500 FP	249 / 196 ✓
Internal: no degradation vs the structural expert alone (upper 95%)	< +0.010	+0.0018 ✓
Internal 90% interval coverage	[0.88, 0.92]	0.902 ✓
Explanation fidelity (relative error)	≤ 1e-9	1e-14 ✓
Held-out: ratio vs NLinear	≤ 1.10	0.839 ✓
Held-out: ratio vs best of NLinear, DLinear, Holt-Winters	≤ 1.15	0.839 ✓
Held-out: ratio vs the previous dense-window variant (~1000 FP)	≤ 1.10	0.773 ✓
Start-point sensitivity, 10 starts (median / max)	≤ 1.10 / ≤ 1.50	1.000 / 1.076 ✓
Tasks with coverage in [0.85, 0.95]	≥ 80%	90% ✓
Numerical divergences	0	0 ✓
Target (non-binding): ratio vs best Chronos	≤ 1.50	1.367 ✓

PRE-REGISTERED DECISION: PROMOTED — all binding criteria met. (The criteria compared against online lightweight models; \$10 shows more demanding comparisons made afterwards.)

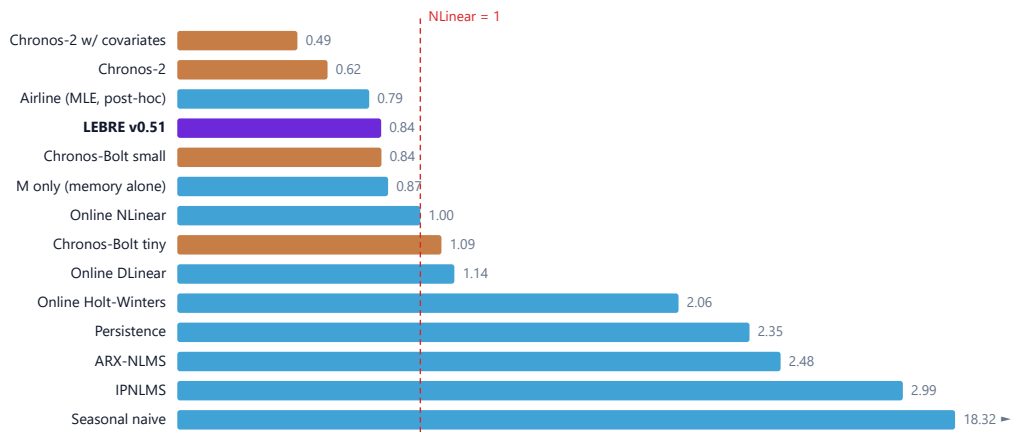
9.4 Results per series (NMSE, lower is better)

	Series	LEBRE v0.51	M only	NLinear	DLinear	Holt-W.	Best Chronos	Persist.	Start	Cover.
Q1	ONS — load of the Brazilian National Grid (2019–20)	0.0114	0.0114	0.0162	0.0167	0.0259	0.0030	0.0707	1.000	0.965
Q2	ONS — hydro generation, Southeast/Center-West (2019–20)	0.0161	0.0161	0.0214	0.0231	0.0331	0.0085	0.0671	1.000	0.937
Q3	ONS — hydro generation, South (2019–20)	0.0150	0.0150	0.0188	0.0200	0.0377	0.0101	0.0447	1.000	0.928
Q4	ONS — wind generation, National Grid (2019–20)	0.0119	0.0119	0.0146	0.0159	0.0280	0.0114	0.0240	1.000	0.922
Q5	Central Bank of Brazil — GBP/BRL (2005–24)	0.0013	0.0013	0.0012	0.0023	0.0014	0.0011	0.0012	1.031	0.881
Q6	Australian electricity demand, series 4	0.0052	0.0052	0.0073	0.0072	0.0452	0.0046	0.0321	1.000	0.851
Q7	Pedestrian counts (Melbourne), series 4	0.0269	0.0269	0.0319	0.0321	0.0365	0.0169	0.1163	1.000	0.920
Q8	Solar power (USA), series 4	0.0078	0.0078	0.0087	0.0113	0.0145	0.0085	0.0115	1.003	0.906
Q9	Air quality (KDD Cup 2018), series 4	0.1008	0.1380	0.0921	0.1021	0.2133	0.0862	0.0934	1.076	0.893
Q10	Australian electricity demand, series 5	0.0066	0.0066	0.0083	0.0085	0.0371	0.0068	0.0385	1.000	0.890

Bold: lowest NMSE among the first six model columns. M only: memory expert alone (post-hoc analysis). Start: max/min NMSE across 10 start points. Cover.: empirical coverage of the 90% interval.

D5: Accuracy on the held-out set (10 new series)

Geometric mean of NMSE relative to the calibrated online NLinear (lower is better)



Chronos-2 w/ covariates: the 4 multivariate series only. Airline (MLE): post-hoc analysis, 9 series (out of memory with $s = 144$).

Geometric-mean NMSE ratio relative to NLinear on the held-out set.

Position	Model	Class	Mean rank
1	Chronos-2	foundation	1.50
2	Chronos-2 w/ covariates	foundation	1.75
3	Chronos-Bolt small	foundation	3.50
4	LEBRE v0.51	online lightweight	3.60
5	Online NLinear	online lightweight	4.20
6	Chronos-Bolt tiny	foundation	5.80
7	Online DLinear	online lightweight	6.50
8	Persistence	online lightweight	7.20
9	Online Holt-Winters	online lightweight	8.40
10	ARX-NLMS	online lightweight	8.50
11	IPNLMS	online lightweight	9.60
12	Seasonal naive	online lightweight	11.33

Correct reading of the ranking: LEBRE v0.51 ranks 4th overall, behind only three foundation models, and is the **best online lightweight model** among the pre-registered comparators. In geometric mean it is $\sim 1.37\times$ behind Chronos-2, at $\sim 10^6\text{--}10^7$ times lower cost, and ties with Chronos-Bolt small (D5). Earlier LEBRE versions are not in the ranking. The airline model fitted per series (post-hoc, §10) is not in the ranking because it was not evaluated under the same conditions (it failed on one series).

9.5 Metrics relative to persistence

$Skill = 1 - MSE/MSE_{persistence}$ (positive = better than repeating the last value); relative MAE = $MAE/MAE_{persistence}$. Per-series values summarised by the median.

Model	Median skill	Series with skill > 0	Median relative MAE
LEBRE v0.51	0.712	8/10	0.542
M only (memory alone)	0.712	8/10	0.542
Airline (MLE, post-hoc)	0.720	8/9	0.515
Online NLinear	0.630	9/10	0.619
Online DLinear	0.604	8/10	0.638
Online Holt-Winters	-0.066	5/10	1.086
Chronos-Bolt small	0.713	9/10	0.562
Chronos-2	0.798	10/10	0.436

On the two exchange-rate series no model beats persistence by a margin: daily exchange rates behave like a random walk.

9.6 Internal suite (input-driven systems)

14 synthetic processes with known structure — exact and multiple delays, latent states, quiescence, regime switches, redundant structure and a non-linear control; 30 fresh seeds per task:

Variant	NMSE	FP/step (mean)	FP peak	Coverage	Exact structure
LEBRE v0.51	0.2151	133.6	249	0.902	90.9%
Structural expert alone	0.2151	99.8	712	0.901	90.8%
Dense-window variant	0.1954	940.1	2,065	0.902	90.8%

Here LEBRE matches the structural expert alone. On this internal suite, used during development, the aggregate exact-structure metric was ~91%. Later analyses with additional diagnostic processes (§11.2) showed that this average does not generalise to every kind of structure: recovery is robust for single delays, partial for several delays and fails in the tested case of an input-driven latent state. The memory alone is **useless** on these systems (NMSE $\approx$ 1.0 on development data).

Secondary checks (sets seen in earlier stages, hence *not* held-out): on a second set of 10 series (5 Brazilian), already used to evaluate an earlier version and with two series that are other stretches of series in the benchmark below, 0.80× NLinear and 1.12× the best Chronos; on a benchmark of 10 real tasks with 36 models (adaptive filters, online networks and reservoirs, online trees, modern linear models and foundation models), 0.85× NLinear, 1.33× the best Chronos and maximum start-point sensitivity 1.16.

9.7 Design evidence (ablations on development data)

13 previously used real series and the internal suite with development seeds. "Real series" = geometric NMSE ratio against an online NLinear with a dense window.

Variant	Real series	Internal	Conclusion
S only (structural expert alone)	2.07	0.221	without M, not competitive on real series
M only (memory alone)	0.96	1.008	without S, useless on input-driven systems
M without seasonal profile G	1.12	—	G is M's decisive ingredient
cascade M → S (no combiner)	0.97	0.304	combiner needed
S → M on S's residual	1.47	0.255	idem
regression combination	0.94	0.240	dynamic averaging preferred
without S dormancy	0.98	0.220	dormancy: -25 FP, no loss
LEBRE v0.51	0.98	0.220	—

Note that "M only" (0.96) is slightly better than v0.51 (0.98) on the real development series: the structural expert does not help there. It is kept only because, without it, the model fails on input-driven systems.

10. LEBRE versus the memory alone and strong classical baselines

All analyses in this section are **post-hoc** (done after the pre-registered evaluation) and cover the main held-out set and the second set (already seen; \$9.6), 20 series in total.

10.1 How much of the result comes from the memory

	LEBRE v0.51	M only	Airline (MLE)	ETS (MLE)	RLS	mean w_S	max w_S	time with $w_S > 0.5$
Q1	0.0114	0.0114	0.0080	0.0228	0.0110	0.000	0.000	0%
Q2	0.0161	0.0161	0.0139	0.0295	0.0157	0.000	0.000	0%
Q3	0.0150	0.0150	0.0139	0.0271	0.0145	0.000	0.000	0%
Q4	0.0119	0.0119	0.0111	0.0177	0.0113	0.000	0.053	0%
Q5	0.0013	0.0013	0.0012	0.0012	0.0012	0.443	1.000	43%
Q6	0.0052	0.0052	0.0039	—	—	0.000	0.000	0%
Q7	0.0269	0.0269	0.0326	0.0449	—	0.000	0.000	0%
Q8	0.0078	0.0078	—	0.0090	—	0.000	0.000	0%
Q9	0.1008	0.1380	0.1384	0.1846	0.0862	0.556	1.000	54%
Q10	0.0066	0.0066	0.0067	0.0188	—	0.000	0.000	0%

NMSE on the main held-out set. w_S : weight of the structural expert along the test segment. "—": the method failed (out of memory for the airline with $s = 144$; RLS or ETS divergence).

- **LEBRE and the memory alone give identical results on 8 of 10 series** ($w_S \approx 0$ throughout the test). The geometric ratio M only / LEBRE is 1.034 on the main held-out set and 1.006 on the second set.
- Across the 20 series, 15 had no structural promotion at all, and in 16 the mean weight of S stayed below 0.01.
- The structural expert only gains weight on **exchange rates** and **air quality**, where it improves on the memory (e.g. 0.101 vs 0.138 on air quality). On the four ONS series with $d = 6$ inputs, w_S never exceeds ~ 0.05 : the other energy-balance variables, observed at $t-1$, add no useful information about the target beyond its own past.
- **Conclusion:** on the real series evaluated, LEBRE's accuracy is essentially that of the memory expert, which is an online form of the airline / Holt-Winters model with weights learned by NLMS.

10.2 Strong classical baselines

Model (geometric ratio vs LEBRE v0.51)	Main held-out	valid series	Second set	valid series
M only (memory alone)	1.034	10/10	1.006	10/10
Airline SARIMA(0,1,1)(0,1,1) _s by MLE	0.948	9/10	0.896	9/10
Additive ETS by MLE (AIC)	1.651	9/10	1.657	10/10
RLS ($\lambda = 0.999$) on M's 5 features	0.937	6/10	0.932	7/10

- **The airline model fitted by maximum likelihood is more accurate than LEBRE** (ratios 0.948 and 0.896). It received the same advantage as the other comparators: θ and Θ were estimated on the first min(15%, 5000) steps of each series and then kept fixed. In recursive form, the airline forecast costs a few dozen operations per step. What LEBRE offers in exchange is not needing this per-series fit, also working on input-driven systems, producing adaptive intervals and logging the structure.
- Additive ETS fitted by maximum likelihood was **worse** than LEBRE and diverged on one series: it is not a robust comparator in this protocol.
- RLS with forgetting on the same 5 features was $\sim 7\%$ more accurate when stable, but **diverged on the long series** (50,000 steps). This shows that the merit lies mainly in the choice of features; NLMS trades some accuracy for numerical stability and $O(k)$ cost.

10.3 What distinguishes LEBRE from the memory alone

	M only	LEBRE v0.51
Accuracy on real seasonal series	same	same ($\leq 3\%$ better, thanks to 2 series)
Input-driven systems (delays, hidden states)	fails (NMSE ≈ 1)	works (NMSE 0.215)
Explanation	additive, 5 fixed terms	additive, with evidence-selected, named structure
Log of structural changes	no	yes (promotions and loss-of-usefulness signals)
Weight indicating the current hypothesis	no	w_S
Cost per step	~ 47 FP	~ 121 FP

Honest positioning. If one only needs to forecast seasonal series without relevant inputs, the memory alone — or an airline model fitted per series — is enough and cheaper. LEBRE is justified when the same forecaster, with no per-series tuning, must serve both seasonal series and input-driven systems, and when the discovered structure and its log have operational value. That last part still needs evidence on real, genuinely input-driven series (§12).

11. Statistical guarantees and reach of structure discovery

11.1 False promotions and evictions on long streams

The guarantees in §5.3 hold under idealised assumptions and per test. To see what happens on long streams, we measured (post-hoc) the structural expert on streams of 100,000 steps, 10 seeds each:

Stream	Total steps	False promotions	Evictions of the true atom
Gaussian null (inputs independent of the target)	1,000,000	0	—
heavy-tailed null (Student t, 3 d.o.f.)	1,000,000	0	—
stationary true structure ($y = 0.8 \cdot x_1(t-3) + \text{noise}$)	1,000,000	—	7

- **Promotions:** no false promotion in 2 million null steps. The control is formally per episode, but accumulation over long streams was negligible in these tests.
- **Evictions:** on a stationary stream the true atom was evicted 7 times in 1 million steps (and later re-promoted). Because of the parsimony rent there is no formal false-alarm-rate guarantee: evictions should be read as *loss-of-usefulness signals*, not as change alarms with a controlled error.

11.2 Reach of structure discovery (synthetic diagnostics)

To delimit what structure discovery can and cannot do, the frozen v0.51 code was run (post-hoc) on synthetic processes with known structure: $d = 5$ independent Gaussian inputs, unit noise, 20,000 steps (unless stated) and 10 seeds each. *Exact structure* is the fraction of checks (every 100 steps, from step 1000) at which the active set equals the true structure; latency is the median, across seeds, of the step of each true atom's first promotion.

Process	Exact structure	1st promotion (steps)	Remark
Two nulls (Gaussian noise; Student t with 3 d.o.f.), 100,000 steps	—	—	0 false promotions in 17,686 test episodes (95% upper bound: $1.7 \cdot 10^{-4}$ per episode, below $\alpha/p = 3.0 \cdot 10^{-4}$)
One delay: $0.8 \cdot x_1(t-3)$, 100,000 steps	98%	1,680	3 evictions over 10 seeds, all followed by re-promotion
One long delay: $0.8 \cdot x_1(t-12)$	93%	2,255	reliable once discovered
Three delays: $0.8 \cdot x_0(t-3) + 0.6 \cdot x_2(t-7) + 0.5 \cdot x_4(t-20)$	57%	2,565 / 2,990 / 8,635	the weakest atom takes long; sensitive to the prior τ
Delay + input-driven latent state: $x_3(t-12) + z(t)$, $z(t) = 0.8 \cdot z(t-1) + x_0(t) + \text{noise}$	0%	2,820 / never	the delay is found; the correct latent, never
Three delays with heavy-tailed noise (t, 3 d.o.f.)	27%	2,635 / 3,285 / 4,705	30 evictions of true atoms over 10 seeds

- **One delay:** reliable. False-promotion control and the eviction rate of §11.1 are confirmed.
- **Latency:** the first promotion typically takes 1.5–3 thousand steps, but the test itself needs only ~200–300 steps once the candidate is proposed. The time is spent in the *search*: the 4 test slots are occupied by candidates without signal until futility (up to 400 steps each), and screening probes each delay only every ~80 steps.
- **Several delays:** partial. The result depends on the mixture prior τ : with $\rho = 1/50, 1/200$ and $1/1000$, exact structure becomes 58%, 28% and 0%.
- **Input-driven latent state:** fails on every seed, for every τ tested. In 10 of 10 seeds, the first latent promoted has pole 0 or 0.5, not 0.8. Since only one latent can be active and the drive inherits its pole, the correct structure, although representable, becomes unreachable. The atom budget is not the bottleneck: it was full in only 3.5% of decisions, and no promotion was blocked. The internal suite (§9.6) contains latent states that are recovered; this process shows that this does not extend to every input-driven latent.
- **Heavy tails:** the CUSUM uses a Gaussian likelihood and frequently evicts true atoms when there are extreme values.
- **Cost of combination:** on every process LEBRE's error is within 0.3% of the structural expert alone, except on the latent process (1.020×): when S is misspecified, the combiner costs ~2%.

12. What this document demonstrates and what it does not

This document contains, in practice, **two distinct results**, and it is important not to confuse them:

Result 1 — a cheap, initialisation-robust, well-evaluated online forecaster

Rigorously demonstrated on real data: pre-registration, never-seen held-out set, calibrated comparators, start-point sensitivity and coverage. The accuracy comes mainly from the memory expert, built from classical pieces (airline model, exponential smoothing, NLMS, dynamic model averaging). The value is **engineering**: a specific combination, with no per-series tuning, sized for ~120 operations per step and with adaptive intervals. **There is no scientific novelty** in this part, and an airline model fitted series by series is somewhat more accurate.

Result 2 — online structure discovery with statistical tests

The structural expert decides, by martingale-based tests and predictable gates (silence, quarantine, dormancy), which delays and hidden states enter the model. It is the most interesting part scientifically and connects to the area of anytime-valid inference (Ramdas et al., 2023). **Today it has evidence only on synthetic processes, and even there with limited reach**: it is reliable for one delay, partial for several delays and fails with an input-driven latent state (§11.2). On the real series evaluated, the structural expert is almost never selected. Defending it would require real, genuinely input-driven series — industrial processes, hydraulic systems with upstream flow, system identification.

Consequence: the statement "LEBRE is competitive on real series" is true, but it should be credited to the memory expert. The statement "LEBRE's structure discovery works on real data" is **not yet demonstrated**, and on synthetic data it is demonstrated only for single delays.

13. Evolution of the architecture and lessons

Earlier versions are mentioned only to explain how the current architecture was reached; the reader does not need to know them.

Stage	Central idea	Outcome
Original organisation	Base path, sparse delays, latent state, lifecycle with shadow probation.	frozen canonical reference
Decisions by statistical tests	Every structural decision becomes a test (mixture martingale, CUSUM); latent in Kalman innovations form; ≤ 100 FP.	stable, but 2–3× worse than modern models on seasonal series
Structural expert alone	Proportionate learning, intervals, exact explanation, input-contract guard.	start-point-dependent instability; ~2.6× NLinear on real data
Dense-window variant	Three experts, window of 96–288 lags.	competitive and stable, but ~1000 FP/step
v0.51	Two minimal experts (structural + airline memory) and dynamic model averaging.	~120 FP/step; accuracy coming mainly from the memory

Lessons. (1) On real seasonal series, accuracy depends on memory of the target itself expressed relative to the last value; the sparse structural hypothesis alone is not enough. (2) Learning the level through the bias term makes early structural decisions depend on the start point. (3) A well-specified memory with 5 learned numbers beats a dense window with hundreds of weights learned online — in line with the M competitions: simple, well-specified statistical methods are hard to beat (Makridakis et al., 2020). (4) A classical model fitted series by series remains a demanding reference.

14. Limits and claims we do not make

- LEBRE does not aim to be the most accurate model: Chronos-2 is ~1.37× better and an airline model fitted series by series is ~5–10% better.
- On the real series evaluated, the result comes almost entirely from the memory expert; structure discovery has evidence only on synthetic data.
- Structure discovery on synthetic data: reliable for one delay; partial for several delays (and sensitive to the prior τ); fails with an input-driven latent state, because only one latent can be active and the first pole to accrue evidence takes the slot; fragile with

heavy-tailed noise. The first promotion typically takes 1.5–3 thousand steps.

- The 150 FP/step budget holds for up to ~6 inputs; every extra input costs ~8 FP. Input standardisation (~12 FP per input) is not included. Execution on microcontrollers was not verified.
- State memory grows with the seasonal period (~12 bytes per phase) and with the number of inputs (~196 bytes per input): ~1.3 KB is the held-out median; with $s = 1440$ it is ~18 KB.
- The interval is updated every 8 steps with a single error indicator; when the period is a multiple of 8, per-series coverage lies between ~0.85 and ~0.97 instead of ~0.90 (§5.5).
- The memory has no numerical regularisation: series with long, exactly constant stretches followed by jumps can make it diverge (§5.2).
- "Start-point sensitivity" measures dependence on initialisation, not stability across different stretches of the series.
- Promotion error control holds per test episode; evictions have no guaranteed false-alarm rate.
- The class is linear in the parameters; one-step-ahead forecasting; long-run, not conditional, interval coverage.
- Exact decomposition alone is not a differentiator: any linear additive model has it.
- We do not claim: state of the art, scientific novelty, physical causality, energy efficiency (FP is not energy) or conditional coverage.

15. Parameters

All values are design constants, identical for every series; none was tuned on the evaluation data.

Parameter	Value	Role
μ (S)	0.1	NLMS step of the structural expert
μ (M)	0.05	NLMS step of the memory
a	0.1	seasonal profile ($\Theta = 0.9$ in the airline model)
mean factor	0.01	exponential mean for mean reversion
η	1/2	derived from the Gaussian likelihood
λ	0.99	forgetting of the combiner and combined σ^2 (~100 steps)
dormancy threshold	0.01	weight below which S's evidence pauses
L	32	maximum candidate delay
M_max	4	structural atom budget
α	0.05	false-promotion rate per episode (threshold $\log(p/\alpha)$)
ARL	6000	$h = \log$ ARL (guarantee only without rent)
T_idle	500	parsimony rent $r = h/T_idle$
poles	{0, 0.5, 0.8, 0.95}	latent-state bank; at most 1 active latent
ρ	1	mixture prior: $\tau = \rho/(\sigma^2 \cdot m_\varphi)$, unit information
T_max	200 samples	test futility (1 sample every 2 steps)
screening	2 per step; weight 0.2	delays probed per step; exponential average of the score
test slots	4	delays under simultaneous test; decisions every 10 steps
replacement	$R \geq h/2$	with a full budget, replaces the atom with the largest CUSUM if $\geq h/2$
ϵ (S / M)	10^{-6} / —	NLMS regularisation (M has none; skips only if $\ \varphi\ ^2 = 0$)
standardisation	10^{-4}	factor of the exponential input means and variances (environment)
clip	± 8	input contract (P6)
excitation	10%	minimum power to accrue evidence (P4)
interval	$\alpha = 0.1$; $\gamma = 0.1$	target coverage 90%; tracker step
cadence	8	steps between square root, inverse, exp and quantile recomputation (quantile with the current-step indicator $\times 8$)
s	from environment	declared sampling period (only external information)

16. References

1. Angelopoulos, A. N., Candès, E. J., Tibshirani, R. J. (2023). Conformal PID control for time series prediction. *NeurIPS* 36.
2. Ansari, A. F. et al. (2024). Chronos: learning the language of time series. *Transactions on Machine Learning Research*.
3. Ansari, A. F. et al. (2025). Chronos-2: from univariate to universal forecasting. *arXiv preprint*.
4. Benesty, J., Gay, S. L. (2002). An improved PNLMS algorithm. *Proc. IEEE ICASSP*, 1881–1884.
5. Box, G. E. P., Jenkins, G. M. (1970). *Time Series Analysis: Forecasting and Control*. Holden-Day.
6. Cesa-Bianchi, N., Lugosi, G. (2006). *Prediction, Learning, and Games*. Cambridge University Press.
7. de la Peña, V. H. (1999). A general class of exponential inequalities for martingales and ratios. *Annals of Probability* 27(1), 537–564.
8. Freund, Y., Schapire, R. E., Singer, Y., Warmuth, M. K. (1997). Using and combining predictors that specialize. *Proc. ACM STOC*, 334–343.
9. Gibbs, I., Candès, E. J. (2021). Adaptive conformal inference under distribution shift. *NeurIPS* 34.
10. Godahewa, R., Bergmeir, C., Webb, G. I., Hyndman, R. J., Montero-Manso, P. (2021). Monash time series forecasting archive. *NeurIPS Datasets and Benchmarks*.
11. Granger, C. W. J., Ramanathan, R. (1984). Improved methods of combining forecasts. *Journal of Forecasting* 3(2), 197–204.
12. Hampel, F. R., Ronchetti, E. M., Rousseeuw, P. J., Stahel, W. A. (1986). *Robust Statistics: The Approach Based on Influence Functions*. Wiley.

13. Harvey, A. C. (1989). *Forecasting, Structural Time Series Models and the Kalman Filter*. Cambridge University Press.
14. Hassibi, B., Sayed, A. H., Kailath, T. (1996). H_∞ optimality of the LMS algorithm. *IEEE Transactions on Signal Processing* 44(2), 267–280.
15. Haykin, S. (2002). *Adaptive Filter Theory*, 4th ed. Prentice Hall.
16. Herbster, M., Warmuth, M. K. (1998). Tracking the best expert. *Machine Learning* 32(2), 151–178.
17. Holt, C. C. (1957/2004). Forecasting seasonals and trends by exponentially weighted moving averages. *International Journal of Forecasting* 20(1), 5–10.
18. Howard, S. R., Ramdas, A., McAuliffe, J., Sekhon, J. (2021). Time-uniform, nonparametric, nonasymptotic confidence sequences. *Annals of Statistics* 49(2), 1055–1080.
19. Hyndman, R. J., Koehler, A. B., Ord, J. K., Snyder, R. D. (2008). *Forecasting with Exponential Smoothing: The State Space Approach*. Springer.
20. Lin, S., Lin, W., Hu, X., Wu, W., Mo, R., Zhong, H. (2024). CycleNet: enhancing time series forecasting through modeling periodic patterns. *NeurIPS* 37.
21. Lin, S., Lin, W., Wu, W., Chen, H., Yang, J. (2024). SparseTSF: modeling long-term time series forecasting with 1k parameters. *ICML*.
22. Lorden, G. (1971). Procedures for reacting to a change in distribution. *Annals of Mathematical Statistics* 42(6), 1897–1908.
23. Makridakis, S., Spiliotis, E., Assimakopoulos, V. (2020). The M4 competition: 100,000 time series and 61 forecasting methods. *International Journal of Forecasting* 36(1), 54–74.
24. Moustakides, G. V. (1986). Optimal stopping times for detecting changes in distributions. *Annals of Statistics* 14(4), 1379–1387.
25. Muth, J. F. (1960). Optimal properties of exponentially weighted forecasts. *Journal of the American Statistical Association* 55(290), 299–306.
26. Nagumo, J., Noda, A. (1967). A learning method for system identification. *IEEE Transactions on Automatic Control* 12(3), 282–287.
27. Page, E. S. (1954). Continuous inspection schemes. *Biometrika* 41(1/2), 100–115.
28. Raftery, A. E., Kárný, M., Ettler, P. (2010). Online prediction under model uncertainty via dynamic model averaging: application to a cold rolling mill. *Technometrics* 52(1), 52–66.
29. Ramdas, A., Grünwald, P., Vovk, V., Shafer, G. (2023). Game-theoretic statistics and safe anytime-valid inference. *Statistical Science* 38(4), 576–601.
30. Seabold, S., Perktold, J. (2010). statsmodels: econometric and statistical modeling with Python. *Proc. 9th Python in Science Conference*.
31. Rissanen, J. (1978). Modeling by shortest data description. *Automatica* 14(5), 465–471.
32. Rudin, C. (2019). Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence* 1, 206–215.
33. Smyl, S. (2020). A hybrid method of exponential smoothing and recurrent neural networks for time series forecasting. *International Journal of Forecasting* 36(1), 75–85.
34. Ville, J. (1939). *Étude critique de la notion de collectif*. Gauthier-Villars.
35. Vovk, V. (1990). Aggregating strategies. *Proc. 3rd Annual Workshop on Computational Learning Theory*, 371–383.
36. Winters, P. R. (1960). Forecasting sales by exponentially weighted moving averages. *Management Science* 6(3), 324–342.
37. Xu, Z., Zeng, A., Xu, Q. (2024). FITS: modeling time series with 10k parameters. *ICLR*.
38. Zeng, A., Chen, M., Zhang, L., Xu, Q. (2023). Are transformers effective for time series forecasting? *AAAI*, 11121–11128.
39. Operador Nacional do Sistema Elétrico (ONS). Dados abertos: balanço de energia por subsistema, curva de carga horária.
40. Banco Central do Brasil. Sistema Gerenciador de Séries Temporais (SGS): taxas de câmbio.